

ProjectWork CV - Deep Image Restoration

Università degli Studi di Ferrara

Dipartimento di Ingegneria, Matematica e Informatica

Professore:

Prof. **Michele FRACCAROLI**

Laureando:

Dott. **Enrico FAVALE**



**Università
degli Studi
di Ferrara**

Il processo di compressione

La compressione delle immagini è una tecnica fondamentale per **ridurre la quantità di dati** necessari per rappresentare un'immagine, consentendo una memorizzazione e trasmissione più efficienti.

Il processo di compressione implicitamente comporta una **perdita di informazione**, poiché l'immagine compressa non è identica all'originale, ma è una rappresentazione approssimata.

A livello visivo una compressione di tipo **lossy** può comportare problematiche del tipo:

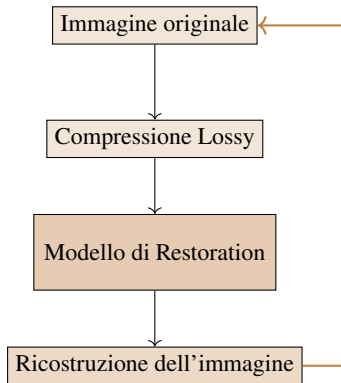
- **Blurring**: perdita di dettagli fini, con conseguente sfocatura dell'immagine.
- **Blocking**: artefatti a blocchi, soprattutto in compressioni a blocchi come JPEG, dove si notano discontinuità tra i blocchi.
- **Ringings**: artefatti a forma di anello intorno a bordi o dettagli ad alto contrasto, causati da oscillazioni nella rappresentazione compressa.

Problema e Pipeline

Il problema affrontato in questo progetto è quello della **restaurazione di immagini** degradate da processi di compressione.

L'obiettivo è sviluppare un modello di deep learning in grado di **ricostruire** l'immagine originale a partire da una versione compressa, migliorando la qualità visiva e riducendo gli artefatti introdotti dalla compressione.

Si può effettuare una ricostruzione di alta qualità di immagini degradate da compressione lossy utilizzando modelli di deep learning?



Dataset di training

Il dataset di training è il **BSD500** (The Berkeley Segmentation Dataset and Benchmark) contenete 500 immagini naturali di alta qualità.

Caratteristiche principali:

- **500 immagini** di scene naturali diverse
- Diverse categorie: paesaggi, oggetti, texture, volti
- Ampiamente utilizzato in ricerca di computer vision
- Pubblicamente disponibile e ben documentato

Per standardizzare le immagini del dataset è stato necessario effettuare un processo di **resizing** alla risoluzione fissa di 256×256 .

Tecniche di compressione

Idea di base:

- Trasformare l'immagine da **spazio dei pixel** a **spazio delle frequenze**
- Ogni frequenza rappresenta una componente di “dettaglio”
- Basse frequenze = struttura generale (bordi, forme)
- Alte frequenze = dettagli fini (texture, rumore)

Compressione con Fourier:

- Applicare un **filtro passa-basso** nello spazio delle frequenze
- Scartare le alte frequenze (dettagli fini)
- Trasformata inversa per tornare allo spazio pixel
- Risultato: immagine compressa ma **sfocata**

Idea di base:

- Decomposizione **multi-scala** dell'immagine
- Rappresenta l'immagine come combinazione di “onde” a diverse scale
- Ogni livello cattura dettagli a quella scala specifica
- Localizzata sia nello spazio che nelle frequenze

Compressione con Wavelet:

- Decomposizione in 4 componenti: **LL, LH, HL, HH**
- **LL** (Low-Low): componente grossolana (importante)
- **LH, HL, HH** (High): dettagli a diverse direzioni
- Scartare i dettagli High per compressione
- Risultato: meno blur, più **blocchi localizzati**

Per entrambe le tecniche di compressione si possono variare i parametri per controllare il livello di compressione. Di seguito sono riportati i parametri utilizzati per la compressione delle immagini del dataset.

Fourier:

- **Keep Fraction:** 25% (mantiene solo un quarto delle frequenze più basse)

Wavelet:

- **Tipo di wavelet:** Haar (una delle wavelet più semplici)
- **Livello di decomposizione:** 1
- **Soglia di compressione:** 20.0 (scarta i dettagli con coefficiente inferiore a 20.0)

Esempi di compressione



Originale



Fourier



Wavelet

Osservazioni:

- La compressione Fourier produce un'immagine più **sfocata**, con perdita di dettagli fini.
- La compressione Wavelet mantiene meglio i bordi e le strutture, ma introduce **blocking**.

L'intero dataset resized a 256×256 viene compresso utilizzando **entrambe** le tecniche (Fourier e Wavelet) per creare due versioni compresse per ciascuna immagine.

Struttura del dataset finale:

- **Originale** → immagine target ad alta qualità
- **Compressa Fourier** → input corrotto con blur diffuso
- **Compressa Wavelet** → input corrotto con blocking localizzato

Motivazione del doppio dataset:

- Addestrare modelli separati per Fourier e Wavelet
- Confrontare le performance della rete sui due tipi di artefatti
- Verificare se un modello generalizza tra i due tipi di compressione

Autoencoder

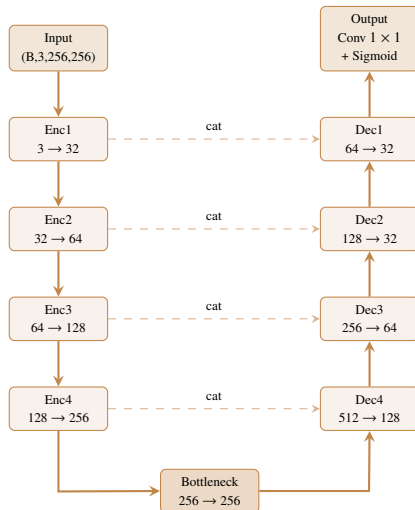
Idea di base:

- Un autoencoder impara a **comprimere** e **ricostruire** un'immagine
- Input: immagine **corrotta** (compressa con Fourier/Wavelet)
- Output: immagine **restaurata** (approssimazione dell'originale)
- La rete non segue regole esplicite: **impara dai dati**

Perché un autoencoder?

- Architettura naturale per trasformazioni immagine $\rightarrow$ immagine
- L'encoder cattura **rappresentazioni astratte** dell'immagine
- Il decoder **ricostruisce** a partire da queste rappresentazioni

Architettura: U-Net Style con Skip Connections



Encoder - Downsampling:

- **Stride=2** (learnable) al posto di MaxPool
- **BatchNorm2d**: stabilizza training, accelera convergenza
- **ReLU**: non-linearità, elimina negativi
- **base_channels=32**: trade-off parametri/qualità

Bottleneck - Rappresentazione Latente:

- Conv2d **senza downsampling** ($16 \times 16 \rightarrow 16 \times 16$)
- 256 canali: massima capacità rappresentativa

Decoder + Skip Connections:

- ConvTranspose2d per upsampling learnable
- `torch.cat`: concatena feature dell'encoder con quelle del decoder
- Recupera dettagli spaziali persi nel downsampling

Layer finale:

- Conv2d(32 $\rightarrow$ 3, kernel=1 $\times$ 1): ricombina le feature in 3 canali RGB
- Sigmoid(): normalizza l'output in $[0, 1]$ (coerente con input normalizzato)
- Output shape: (B, 3, 256, 256) - identica all'input

Parametri del modello (base_channels=32):

Componente	Parametri
Encoder (4 stage)	~560K
Bottleneck	~590K
Decoder (4 stage)	~1.1M
Output layer	~1K
Totale	~2.3M

Loss Function: RestorationLoss

Loss combinata: MSE + Perceptual Loss

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{MSE}} + \beta \cdot \mathcal{L}_{\text{perceptual}}$$

MSE (pixel-wise):

- Misura errore pixel per pixel tra output e target
- Garantisce fedeltà numerica
- Problema noto: tende a produrre output **sfocati** se usato da solo

Perceptual Loss (VGG16):

- Estrae feature dai primi 16 layer di VGG16 pre-trainato su ImageNet
- Confronta le feature di output e target (non i pixel)
- Cattura somiglianza **strutturale e percettiva**
- VGG16 **frozen**: pesi mai aggiornati durante training
- Pesi usati: $\alpha = 1.0$, $\beta = 0.1$

Ottimizzatore e Scheduler:

- **Adam:** $lr=10^{-3}$, $weight_decay=10^{-4}$
- **CosineAnnealingLR:** lr decresce gradualmente da 10^{-3} a 10^{-6}
- Evita oscillazioni a fine training

Iperparametri:

- Epochs: **100**
- Batch size: **4**
- Image size: 256×256
- Split: 70/15/15

Checkpointing: Salva il modello con **minima val loss**

Training Separato - Fourier vs Wavelet

Perché Training Separato?

Idea di base:

- Fourier e Wavelet introducono **artefatti diversi**
- Fourier: blur diffuso, halo attorno ai bordi
- Wavelet: blocking localizzato, artefatti concentrati
- Una singola rete potrebbe fare compromessi su entrambi

Strategia scelta:

- Addestrare **due modelli separati**
- Modello 1: specializzato su immagini compresse con **Fourier**
- Modello 2: specializzato su immagini compresse con **Wavelet**
- Permette di **confrontare le performance** sui due tipi di artefatti

Vantaggio:

- Ogni rete impara una **strategia ottimale** per il suo tipo di compressione
- Migliore specializzazione = migliori risultati complessivi

Modifica al Trainer:

Nel codice, il parametro `train_restriction` controlla il dataset:

Valore	Dataset	Architettura
None	Fourier + Wavelet	Stessa rete su entrambi
"fourier_only"	Solo Fourier	Modello specializzato
"wavelet_only"	Solo Wavelet	Modello specializzato

L'intera logica di filtraggio avviene nella selezione esclusivamente delle immagini necessarie.

Restormer

Restoration Transformer - Zamir et al., CVPR 2022 - arxiv.org/abs/2111.09881

Problema: i Transformer classici calcolano l'attenzione **tra pixel**, con complessità $O(H^2W^2)$: inapplicabile su immagini ad alta risoluzione.

Soluzione di Restormer: calcolare l'attenzione **tra canali** invece che tra pixel.

- Complessità ridotta a $O(C^2)$: scalabile a qualsiasi risoluzione
- Cattura comunque dipendenze **globali** nell'immagine
- State-of-the-art su denoising, deblurring, deraining (CVPR 2022)

Struttura: encoder-decoder gerarchico a 4 livelli (stile U-Net) con skip connections.

Ogni livello contiene TransformerBlock, composto da:

- **MDTA** (Multi-DConv Head Transposed Attention):
self-attention tra canali + depthwise conv per contesto spaziale locale
- **GDFN** (Gated-DConv Feed-Forward Network):
due branch paralleli con gating per controllare il flusso di informazione

Residual learning:

$$\hat{I} = \text{net}(x) + x$$

La rete impara la **differenza** tra immagine corrotta e originale, non l'immagine intera.

Perché Restormer come secondo modello?

- Confrontare approccio **CNN-based** (ConvAutoencoder) con **Transformer-based**
- Valutare se la maggiore capacità porta benefici concreti

Configurazione: parametri originali del paper

Parametro	Valore
Base channels (dim)	48
Blocks per level	[4, 6, 6, 8]
Attention heads	[1, 2, 4, 8]
Totale parametri	~26M vs ~2.3M del ConvAE

Integrazione: wrapper con interfaccia identica al ConvAutoencoder - stesso Trainer, stessa Loss, stesso dataset, zero modifiche al codice.

Valutazione delle Performance

PSNR (Peak Signal-to-Noise Ratio):

- Misura la fedeltà **quantitativa** pixel per pixel come rapporto tra immagine originale e distorsione introdotta
- Più alto è meglio: > 30 dB ottimo, > 25 dB accettabile

SSIM (Structural Similarity Index):

- Misura la somiglianza **percettiva**: luminanza, contrasto, struttura
- Valori in $[0, 1]$: più vicino a 1, più simile all'originale
- Più correlato alla percezione umana rispetto al PSNR

Metriche di supporto:

- **LPIPS (AlexNet)**: distanza percettiva tramite feature di rete pre-trainata - più basso è meglio
- **MSE**: errore quadratico medio - più basso è meglio
- **MAE**: errore assoluto medio - più basso è meglio

Risultati: Confronto tra Modelli

Esperimento	PSNR $\uparrow$	Δ PSNR $\uparrow$	SSIM $\uparrow$	Δ SSIM	LPIPS $\downarrow$	Δ LPIPS $\downarrow$
Baseline (compressa)	25.60	-	0.788	-	0.281	-
AE - Both	26.62	+1.02	0.785	-0.003	0.136	-0.145
AE - Fourier only	24.22	-1.38	0.731	-0.057	0.194	-0.087
AE - Wavelet only	26.28	+0.68	0.783	-0.006	0.227	-0.054
Restormer - Both	28.20	+2.60	0.809	+0.021	0.142	-0.139
Restormer - Fourier only	25.51	-0.09	0.752	-0.037	0.169	-0.112
Restormer - Wavelet only	27.57	+1.97	0.810	+0.021	0.238	-0.043

Modello migliore: Restormer trained su entrambe le compressioni

- PSNR: +2.60 dB rispetto alla baseline - miglioramento significativo
- SSIM: +0.021 - unico modello a migliorare la similarità strutturale
- LPIPS: dimezzato ($0.281 \rightarrow 0.142$) - qualità percettiva molto migliorata

Training su entrambe le compressioni vs separato:

- I modelli trained su **entrambe** le compressioni superano quelli separati
- Il dataset misto funge da **augmentation implicita**: più variabilità, migliore generalizzazione
- I modelli **fourier_only** peggiorano rispetto alla baseline: il rumore imparato a ridurre è troppo specifico per la compressione

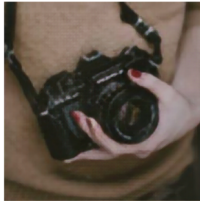
Test visivo di inferenza

AutoEncoder - Inferenza

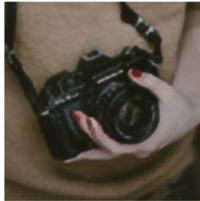
[Fourier keep=0.25]
PSNR 27.04 dB | SSIM 0.7665



All data
PSNR 26.05 dB (+-0.98) | SSIM 0.7532 (+-0.0134)



Fourier only
PSNR 26.29 dB (+-0.75) | SSIM 0.7523 (+-0.0143)



Wavelet only
PSNR 24.28 dB (+-2.76) | SSIM 0.7290 (+-0.0375)



[Wavelet haar -lvl 1, thr 20]
PSNR 25.46 dB | SSIM 0.8353



All data
PSNR 29.27 dB (+3.81) | SSIM 0.8778 (+0.0425)



Fourier only
PSNR 24.59 dB (+-0.87) | SSIM 0.7733 (+-0.0621)



Wavelet only
PSNR 29.71 dB (+4.23) | SSIM 0.8818 (+0.0465)



Restormer - Inferenza

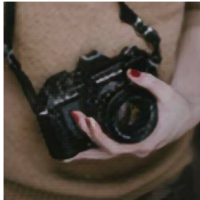
[Fourier keep=0.25]
PSNR 27.04 dB | SSIM 0.7665



All data
PSNR 27.04 dB (+0.00) | SSIM 0.7666 (+0.0000)



Fourier only
PSNR 27.63 dB (+0.59) | SSIM 0.7864 (+0.0198)



Wavelet only
PSNR 24.80 dB (+-2.23) | SSIM 0.7362 (+-0.0303)



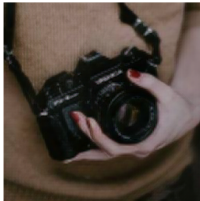
[Wavelet haar -vl 1 thr 20]
PSNR 25.46 dB | SSIM 0.8353



All data
PSNR 30.66 dB (+5.20) | SSIM 0.8979 (+0.0625)



Fourier only
PSNR 25.18 dB (+-0.29) | SSIM 0.7854 (+-0.0499)



Wavelet only
PSNR 30.89 dB (+5.43) | SSIM 0.8996 (+0.0643)



Conclusioni

Obiettivo: rimuovere artefatti di compressione (Fourier/Wavelet) tramite deep learning.

Cosa abbiamo ottenuto:

- **ConvAutoencoder** U-Net style: leggero (2.3M params), competitivo
- **Restormer:** Transformer-based (26M params), migliori performance assolute
- Il modello migliore (*Restormer - both*) ottiene +2.60 dB di PSNR e +0.021 SSIM rispetto alla baseline
- LPIPS dimezzato: qualità percettiva significativamente migliorata

Lezione principale:

- Addestrare su **entrambe** le compressioni supera il training specializzato
- La variabilità nel dataset è più utile della specializzazione
- Un modello CNN ben progettato rimane competitivo con un Transformer

Limiti del progetto:

- Dataset relativamente piccolo (500 immagini BSD500)
- Training su un solo livello di compressione fisso (no generalizzazione su keep_fraction variabile)

Possibili estensioni:

- Addestrare su **livelli di compressione multipli** per maggiore robustezza
- Usare dataset più grandi (DIV2K, ImageNet subset)
- Confronto con metodi classici di deblurring e denoising

Grazie per l'attenzione